

Problem 01: Awaited mock requests

Easy | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Write an async program that simulates simple I/O requests using `asyncio.sleep`. Each request must be awaited and printed in the same order as the input.

INPUT FORMAT

Line 1: integer `n`, the number of requests. Next `n` lines: `request_id delay_ms`. Assume $1 \leq n \leq 20$ and $0 \leq \text{delay_ms} \leq 2000$.

OUTPUT FORMAT

Print one `request_id` per line after its simulated request has completed, preserving input order.

EXAMPLE INPUT

Example input 1

```
3
profile 20
invoice 5
settings 10
```

Expected output 1

```
profile
invoice
settings
```

Example input 2

```
2
healthcheck 0
report 15
```

Expected output 2

```
healthcheck
report
```

Problem 02: Completion order with create_task

Easy | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Create all request coroutines as tasks before awaiting any of them. Print request IDs in the order their simulated delays complete, using input order to break ties.

INPUT FORMAT

Line 1: integer n . Next n lines: request_id delay_ms. Assume $1 \leq n \leq 30$; all delays are integer milliseconds.

OUTPUT FORMAT

Print one request_id per line in completion order.

EXAMPLE INPUT

Example input 1

```
4
auth 30
cache 10
billing 20
logs 10
```

Expected output 1

```
cache
logs
billing
auth
```

Example input 2

```
3
warmup 0
api 5
db 5
```

Expected output 2

```
warmup
api
db
```

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Use `asyncio.gather` to run simulated API calls concurrently. Even though the calls finish at different times, the final report must keep the original input order.

INPUT FORMAT

Line 1: integer `n`. Next `n` lines: `endpoint delay_ms status_code`. Assume endpoint has no spaces and `status_code` is an integer.

OUTPUT FORMAT

For each endpoint, print: `index endpoint status_code`. Indexing starts at 0 and follows the input order.

EXAMPLE INPUT

Example input 1

```
3
/users 30 200
/invoices 5 503
/health 1 200
```

Expected output 1

```
0 /users 200
1 /invoices 503
2 /health 200
```

Example input 2

```
2
/login 15 401
/logout 0 204
```

Expected output 2

```
0 /login 401
1 /logout 204
```

Problem 04: Async uppercase transform

Easy | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Implement a coroutine that waits for the given delay and returns the payload in uppercase. Run all transformations concurrently and print the results in input order.

INPUT FORMAT

Line 1: integer n. Next n lines: user_id delay_ms payload. Payload contains no spaces; $1 \leq n \leq 25$.

OUTPUT FORMAT

Print one line per input record as user_id:UPPERCASE_PAYLOAD.

EXAMPLE INPUT

Example input 1

```
3
u1 30 login
u2 5 invoice
u3 10 export
```

Expected output 1

```
u1:LOGIN
u2:INVOICE
u3:EXPORT
```

Example input 2

```
2
ops 0 ready
sales 8 quote
```

Expected output 2

```
ops:READY
sales:QUOTE
```

Problem 05: Missing await in delayed doubles

Easy | Debugging task

DEBUGGING TASK - code has valid syntax and correct indentation, but contains a logic/runtime bug

TASK

Fix the program so each delayed doubling operation is actually awaited. The corrected version should run the operations concurrently and print the doubled values in input order.

INPUT FORMAT

Line 1: integer n. Next n lines: value delay_ms. Values and delays are integers; $1 \leq n \leq 20$.

OUTPUT FORMAT

Print one doubled value per line in the same order as the input.

EXPECTED BEHAVIOR

The corrected program should await the coroutines, preferably with `asyncio.gather`, so it prints numeric results instead of coroutine objects.

EXAMPLE INPUT

Example input 1

```
3
5 20
8 5
1 10
```

Expected output 1

```
10
16
2
```

Example input 2

```
2
7 0
12 3
```

Expected output 2

```
14
24
```

Broken code

```
import asyncio
import sys
async def double_later(value, delay):
    await asyncio.sleep(delay / 1000)
    return value * 2
async def main():
    n = int(sys.stdin.readline())
    results = []
    for _ in range(n):
        value, delay = map(int, sys.stdin.readline().split())
        results.append(double_later(value, delay))
    for result in results:
        print(result)
    asyncio.run(main())
```

Problem 06: as_completed result stream

Easy | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Start all jobs concurrently and stream their results as each job finishes. Use `asyncio.as_completed` or equivalent task tracking.

INPUT FORMAT

Line 1: integer `n`. Next `n` lines: `job_id delay_ms`. Assume `1 <= n <= 30` and ties finish in input order.

OUTPUT FORMAT

Print `FINISHED job_id` for each job in completion order.

EXAMPLE INPUT

Example input 1

```
3
pdf 40
email 10
cache 25
```

Expected output 1

```
FINISHED email
FINISHED cache
FINISHED pdf
```

Example input 2

```
4
a 5
b 5
c 1
d 9
```

Expected output 2

```
FINISHED c
FINISHED a
FINISHED b
FINISHED d
```

Problem 07: Scheduled timer events

Easy | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Schedule a set of timer events with `asyncio.sleep`. When multiple events have the same delay, preserve their input order in the printed output.

INPUT FORMAT

Line 1: integer `n`. Next `n` lines: `event_name delay_ms`. Assume all delays are non-negative integers.

OUTPUT FORMAT

Print `T+delay_ms event_name` for each event in the order it fires.

EXAMPLE INPUT

Example input 1

```
4
boot 0
metrics 50
ready 20
refresh 20
```

Expected output 1

```
T+0 boot
T+20 ready
T+20 refresh
T+50 metrics
```

Example input 2

```
3
alpha 15
beta 5
gamma 10
```

Expected output 2

```
T+5 beta
T+10 gamma
T+15 alpha
```

Problem 08: Per-task timeout

Easy | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Run all simulated requests concurrently, but apply the same timeout to each request. A request whose delay is greater than the timeout should be reported as TIMEOUT.

INPUT FORMAT

Line 1: timeout_ms n. Next n lines: request_id delay_ms.
Assume $0 \leq \text{timeout_ms} \leq 5000$ and $1 \leq n \leq 30$.

OUTPUT FORMAT

Print request_id OK or request_id TIMEOUT in input order.

EXAMPLE INPUT

Example input 1

```
20 3
auth 5
report 25
cache 20
```

Expected output 1

```
auth OK
report TIMEOUT
cache OK
```

Example input 2

```
0 2
instant 0
slow 1
```

Expected output 2

```
instant OK
slow TIMEOUT
```

Problem 09: Shard count collector

Easy | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Query simulated shards concurrently and collect their counts. Print the total count and the shard with the largest delay.

INPUT FORMAT

Line 1: integer n. Next n lines: shard_id delay_ms count. Assume shard_id has no spaces and counts are non-negative integers.

OUTPUT FORMAT

First print TOTAL total_count. Then print SLOWEST shard_id.

EXAMPLE INPUT

Example input 1

```
3
s1 20 4
s2 5 8
s3 35 2
```

Expected output 1

```
TOTAL 14
SLOWEST s3
```

Example input 2

```
2
eu 10 7
us 10 6
```

Expected output 2

```
TOTAL 13
SLOWEST eu
```

Problem 10: Gather used when completion order is required

Easy | Debugging task

DEBUGGING TASK - code has valid syntax and correct indentation, but contains a logic/runtime bug

TASK

Fix the program so it prints tasks in completion order rather than input order. The code is syntactically valid, but it uses an awaiting pattern that hides the true finish order.

INPUT FORMAT

Line 1: integer n. Next n lines: task_name delay_ms. Assume $1 \leq n \leq 20$ and ties keep input order.

OUTPUT FORMAT

Print one task_name per line in completion order.

EXPECTED BEHAVIOR

The corrected program should consume tasks as they complete, for example with `asyncio.as_completed`, instead of relying on `gather`'s input-order result list.

EXAMPLE INPUT

Example input 1

```
3
slow 30
fast 5
mid 15
```

Expected output 1

```
fast
mid
slow
```

Example input 2

```
3
a 10
b 10
c 1
```

Expected output 2

```
c
a
b
```

Broken code

```
import asyncio
import sys
async def finish(name, delay):
    await asyncio.sleep(delay / 1000)
    return name
async def main():
    n = int(sys.stdin.readline())
    tasks = []
    for _ in range(n):
        name, delay = sys.stdin.readline().split()
        tasks.append(asyncio.create_task(finish(name, int(delay))))
    results = await asyncio.gather(*tasks)
    for name in results:
        print(name)
    asyncio.run(main())
```

Problem 11: Result and error collection

Easy-medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Run all simulated calls concurrently and collect both successful results and failures. A failed call should not stop the other calls from completing.

INPUT FORMAT

Line 1: integer n. Next n lines: call_id delay_ms outcome, where outcome is OK or FAIL. Assume $1 \leq n \leq 40$.

OUTPUT FORMAT

Print call_id OK for successes and call_id ERROR for failures, preserving input order.

EXAMPLE INPUT

Example input 1

```
4
auth 10 OK
billing 5 FAIL
search 1 OK
export 20 FAIL
```

Expected output 1

```
auth OK
billing ERROR
search OK
export ERROR
```

Example input 2

```
2
cache 0 OK
risk 3 FAIL
```

Expected output 2

```
cache OK
risk ERROR
```

Problem 12: Bounded concurrent downloader

Easy-medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Implement a bounded-concurrency worker that starts at most limit jobs at the same time. Jobs are started in input order whenever a slot is free, and output follows completion order.

INPUT FORMAT

Line 1: limit n. Next n lines: job_id delay_ms. Assume $1 \leq \text{limit} \leq n \leq 50$ and ties finish in start order.

OUTPUT FORMAT

Print one completed job_id per line.

EXAMPLE INPUT

Example input 1

```
2 4
A 5
B 10
C 3
D 1
```

Expected output 1

```
A
C
D
B
```

Example input 2

```
1 3
x 2
y 1
z 1
```

Expected output 2

```
x
y
z
```

Problem 13: Single consumer queue

Easy-medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Create producer tasks that put items into an asyncio.Queue after their delay. A single consumer should read from the queue and print items in the order they become available.

INPUT FORMAT

Line 1: integer n. Next n lines: item_id delay_ms. Assume $1 \leq n \leq 40$ and ties keep input order.

OUTPUT FORMAT

Print CONSUMED item_id for each item in queue receive order.

EXAMPLE INPUT

Example input 1

```
4
alpha 30
beta 5
gamma 15
delta 5
```

Expected output 1

```
CONSUMED beta
CONSUMED delta
CONSUMED gamma
CONSUMED alpha
```

Example input 2

```
3
one 0
two 2
three 1
```

Expected output 2

```
CONSUMED one
CONSUMED three
CONSUMED two
```

Problem 14: First successful response

Easy-medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Start all endpoint calls concurrently and report the first endpoint that succeeds. Failed calls do not count; if no endpoint succeeds, print NONE.

INPUT FORMAT

Line 1: integer n. Next n lines: endpoint delay_ms outcome, where outcome is OK or FAIL. Assume ties are broken by input order.

OUTPUT FORMAT

Print FIRST endpoint for the earliest successful endpoint, or NONE if all endpoints fail.

EXAMPLE INPUT

Example input 1

```
4
primary 30 FAIL
replica-a 25 OK
replica-b 5 FAIL
cache 40 OK
```

Expected output 1

```
FIRST replica-a
```

Example input 2

```
3
a 1 FAIL
b 5 FAIL
c 2 FAIL
```

Expected output 2

```
NONE
```

Problem 15: Queue order bug

Easy-medium | Debugging task

DEBUGGING TASK - code has valid syntax and correct indentation, but contains a logic/runtime bug

TASK

Fix the queue type so the consumer receives items in first-in, first-out order. The current program is runnable but uses the wrong queue behavior.

INPUT FORMAT

Line 1: integer n. Next n lines: item_id. Assume $1 \leq n \leq 30$ and item_id has no spaces.

OUTPUT FORMAT

Print one item_id per line in the same order as the input.

EXPECTED BEHAVIOR

The corrected program should use an `asyncio.Queue` so items are consumed in FIFO order.

EXAMPLE INPUT

Example input 1

```
4
login
invoice
email
report
```

Expected output 1

```
login
invoice
email
report
```

Example input 2

```
3
a
b
c
```

Expected output 2

```
a
b
c
```

Broken code

```
import asyncio
import sys
async def main():
    n = int(sys.stdin.readline())
    queue = asyncio.LifoQueue()
    for _ in range(n):
        await queue.put(sys.stdin.readline().strip())
    while not queue.empty():
        item = await queue.get()
        print(item)
        queue.task_done()
    asyncio.run(main())
```

Problem 16: Cancel unfinished jobs

Easy-medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Run all jobs concurrently but cancel any job that has not finished by `cancel_after_ms`. Print completed jobs first, then list the cancelled jobs in input order.

INPUT FORMAT

Line 1: `cancel_after_ms n`. Next `n` lines: `job_id delay_ms`. Assume $1 \leq n \leq 50$.

OUTPUT FORMAT

Print `DONE job_id` for completed jobs in completion order. Then print `CANCELLED job_id` for unfinished jobs in input order.

EXAMPLE INPUT

Example input 1

```
20 4
a 5
b 25
c 10
d 30
```

Expected output 1

```
DONE a
DONE c
CANCELLED b
CANCELLED d
```

Example input 2

```
10 3
x 10
y 0
z 11
```

Expected output 2

```
DONE y
DONE x
CANCELLED z
```

Problem 17: Retry flaky endpoints

Easy-medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Implement retry logic for simulated endpoints. Each endpoint provides a sequence of attempt outcomes, and each attempt takes the same delay.

INPUT FORMAT

Line 1: max_attempts n. Next n lines: endpoint delay_ms outcomes, where outcomes is a string like FFO using F for failure and O for success. Stop after success or after max_attempts.

OUTPUT FORMAT

Print endpoint OK attempts_used or endpoint FAIL attempts_used in input order.

EXAMPLE INPUT

Example input 1

```
3 3
auth 5 F0
billing 5 FFO
export 1 FFF
```

Expected output 1

```
auth OK 2
billing OK 3
export FAIL 3
```

Example input 2

```
2 2
cache 0 0
risk 3 F00
```

Expected output 2

```
cache OK 1
risk OK 2
```

Problem 18: Periodic ticker

Easy-medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Write an async ticker that emits a fixed number of ticks separated by `interval_ms`. The output should use simulated elapsed time rather than wall-clock timestamps.

INPUT FORMAT

Line 1: `interval_ms count`. Assume $0 \leq \text{interval_ms} \leq 5000$ and $1 \leq \text{count} \leq 20$.

OUTPUT FORMAT

For `i` from 1 to `count`, print `TICK i AT elapsed_ms`.

EXAMPLE INPUT

Example input 1

```
100 3
```

Expected output 1

```
TICK 1 AT 100
TICK 2 AT 200
TICK 3 AT 300
```

Example input 2

```
0 4
```

Expected output 2

```
TICK 1 AT 0
TICK 2 AT 0
TICK 3 AT 0
TICK 4 AT 0
```

Problem 19: First K completions

Easy-medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Start every job, collect only the first k completed jobs, and cancel the rest. This is useful when enough parallel results have arrived and remaining work is no longer needed.

INPUT FORMAT

Line 1: k n. Next n lines: job_id delay_ms. Assume $1 \leq k \leq n$ and ≤ 50 and ties use input order.

OUTPUT FORMAT

Print TAKE job_id for the first k completed jobs. Finally print CANCELLED count for all remaining jobs.

EXAMPLE INPUT

Example input 1

```
2 5
a 50
b 10
c 20
d 5
e 30
```

Expected output 1

```
TAKE d
TAKE b
CANCELLED 3
```

Example input 2

```
3 3
x 2
y 1
z 3
```

Expected output 2

```
TAKE y
TAKE x
TAKE z
CANCELLED 0
```

Problem 20: Unawaited gather

Easy-medium | Debugging task

DEBUGGING TASK - code has valid syntax and correct indentation, but contains a logic/runtime bug

TASK

Fix the program so it waits for all delayed values before summing them. The current code creates a gather object but uses it as if the results were already available.

INPUT FORMAT

Line 1: integer n. Next n lines: value delay_ms. Assume $1 \leq n \leq 30$.

OUTPUT FORMAT

Print SUM total after all tasks complete.

EXPECTED BEHAVIOR

The corrected program should await `asyncio.gather` before summing the returned list of integers.

EXAMPLE INPUT

Example input 1

```
3
4 10
5 0
6 2
```

Expected output 1

```
SUM 15
```

Example input 2

```
2
100 1
7 1
```

Expected output 2

```
SUM 107
```

Broken code

```
import asyncio
import sys
async def load(value, delay):
    await asyncio.sleep(delay / 1000)
    return value
async def main():
    n = int(sys.stdin.readline())
    tasks = []
    for _ in range(n):
        value, delay = map(int, sys.stdin.readline().split())
        tasks.append(load(value, delay))
    results = asyncio.gather(*tasks)
    print("SUM", sum(results))
    asyncio.run(main())
```

Problem 21: Batch rate limiter

Medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Simulate a simple async rate limiter that starts at most limit requests per one-second bucket. Requests are considered in input order, and waiting requests move to the next bucket when the current bucket is full.

INPUT FORMAT

Line 1: limit n. Next n lines: request_id arrival_second. Assume arrivals are sorted by arrival_second and $1 \leq \text{limit} \leq 10$.

OUTPUT FORMAT

For each request, print request_id START second, preserving input order after assigning start seconds.

EXAMPLE INPUT

Example input 1

```
2 5
a 0
b 0
c 0
d 1
e 1
```

Expected output 1

```
a START 0
b START 0
c START 1
d START 1
e START 2
```

Example input 2

```
3 4
r1 2
r2 2
r3 2
r4 2
```

Expected output 2

```
r1 START 2
r2 START 2
r3 START 2
r4 START 3
```

Problem 22: Priority queue workers

Medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Use an `asyncio.PriorityQueue` to feed jobs to a fixed number of workers. Lower priority numbers run first; workers become available again after each job's delay.

INPUT FORMAT

Line 1: workers `n`. Next `n` lines: priority job_id delay_ms. Assume all jobs are enqueued before workers start and ties use input order.

OUTPUT FORMAT

Print job_id as each job completes.

EXAMPLE INPUT

Example input 1

```
2 4
5 report 30
1 auth 10
3 invoice 5
2 cache 20
```

Expected output 1

```
auth
invoice
cache
report
```

Example input 2

```
1 3
2 b 1
1 a 5
3 c 1
```

Expected output 2

```
a
b
c
```

Problem 23: Sequential batches, concurrent inside each batch

Medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Process input jobs in batches of size `batch_size`. Jobs inside a batch run concurrently and are printed in completion order, but the next batch cannot start until the current batch is fully done.

INPUT FORMAT

Line 1: `batch_size n`. Next `n` lines: `job_id delay_ms`. Assume $1 \leq \text{batch_size} \leq n \leq 60$.

OUTPUT FORMAT

Print one `job_id` per line in the order batches produce completed work.

EXAMPLE INPUT

Example input 1

```
2 5
a 30
b 10
c 5
d 40
e 1
```

Expected output 1

```
b
a
c
d
e
```

Example input 2

```
3 4
x 9
y 2
z 4
w 1
```

Expected output 2

```
y
z
x
w
```

Problem 24: Per-key async cache

Medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Build an async cache so concurrent requests for the same missing key share one fetch. Requests should return values in input order, and the program should also report how many actual fetches happened.

INPUT FORMAT

Line 1: integer n. Next n lines: key delay_ms value. If several lines use the same key, the first value for that key is the fetched value.

OUTPUT FORMAT

Print key value for each request in input order. Then print FETCHES count, where count is the number of distinct fetched keys.

EXAMPLE INPUT

Example input 1

```
4
user:1 30 Alice
user:1 5 Ignored
plan:7 10 Pro
user:1 1 IgnoredAgain
```

Expected output 1

```
user:1 Alice
user:1 Alice
plan:7 Pro
user:1 Alice
FETCHES 2
```

Example input 2

```
3
a 2 A
b 2 B
a 2 X
```

Expected output 2

```
a A
b B
a A
FETCHES 2
```

Problem 25: Async counter race

Medium | Debugging task

DEBUGGING TASK - code has valid syntax and correct indentation, but contains a logic/runtime bug

TASK

Fix the race condition in the shared counter update. The corrected program should count every task exactly once even though each task yields control during the update.

INPUT FORMAT

Line 1: integer n, the number of increments. Assume $1 \leq n \leq 1000$.

OUTPUT FORMAT

Print COUNT n after all increments complete.

EXPECTED BEHAVIOR

The corrected program should protect the read-modify-write operation, for example with an `asyncio.Lock`.

EXAMPLE INPUT

Example input 1

5

Expected output 1

COUNT 5

Example input 2

20

Expected output 2

COUNT 20

Broken code

```
import asyncio
import sys
counter = 0
async def increment():
    global counter
    current = counter
    await asyncio.sleep(0)
    counter = current + 1
async def main():
    n = int(sys.stdin.readline())
    await asyncio.gather(*(increment() for _ in range(n)))
    print("COUNT", counter)
asyncio.run(main())
```

Problem 26: Deadline-aware steps

Medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Each job consists of several sequential async steps, and the whole job has a deadline. Run jobs concurrently, but a job must stop before starting a step that would finish after its deadline.

INPUT FORMAT

Line 1: integer n. Next n lines: job_id deadline_ms step1,step2,... where each step duration is an integer. Assume steps are comma-separated with no spaces.

OUTPUT FORMAT

For each job in input order, print job_id DONE steps_completed or job_id TIMEOUT steps_completed.

EXAMPLE INPUT

Example input 1

```
3
a 30 10,10,5
b 15 10,10
c 0 0
```

Expected output 1

```
a DONE 3
b TIMEOUT 1
c DONE 1
```

Example input 2

```
2
job1 12 5,5,5
job2 8 2,2
```

Expected output 2

```
job1 TIMEOUT 2
job2 DONE 2
```

Problem 27: Fan-out fan-in pipeline

Medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Implement a two-stage async pipeline: download first, then transform. Downloads are unlimited, but only `transform_limit` transforms may run at once; output each item when its transform finishes.

INPUT FORMAT

Line 1: `transform_limit` `n`. Next `n` lines: `item_id` `download_ms` `transform_ms`. Assume $1 \leq \text{transform_limit} \leq n \leq 50$.

OUTPUT FORMAT

Print `item_id` in final completion order.

EXAMPLE INPUT

Example input 1

```
1 3
a 5 20
b 10 1
c 1 5
```

Expected output 1

```
c
a
b
```

Example input 2

```
2 4
w 0 10
x 0 5
y 3 1
z 4 1
```

Expected output 2

```
x
y
z
w
```

Problem 28: Graceful queue shutdown

Medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Use worker tasks and sentinel values to stop a queue cleanly after all jobs are processed. Assign jobs to the first available worker, with lower worker IDs winning ties.

INPUT FORMAT

Line 1: workers n . Next n lines: job_id delay_ms. Assume jobs are enqueued in input order and $1 \leq \text{workers} \leq 10$.

OUTPUT FORMAT

Print worker_id job_id when a worker finishes that job.

EXAMPLE INPUT

Example input 1

```
2 4
a 5
b 10
c 2
d 1
```

Expected output 1

```
0 a
0 c
0 d
1 b
```

Example input 2

```
3 3
x 4
y 1
z 1
```

Expected output 2

```
1 y
2 z
0 x
```

Problem 29: Per-host concurrency cap

Medium | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Run requests concurrently with a per-host cap of one active request per host. Requests for different hosts may overlap, but requests for the same host must run in input order.

INPUT FORMAT

Line 1: integer n . Next n lines: host request_id delay_ms.
Assume hosts and request IDs contain no spaces.

OUTPUT FORMAT

Print request_id as each request completes; ties use input order.

EXAMPLE INPUT

Example input 1

```
5
api a1 10
api a2 1
cdn c1 3
db d1 5
api a3 1
```

Expected output 1

```
c1
d1
a1
a2
a3
```

Example input 2

```
3
h1 x 2
h2 y 2
h1 z 1
```

Expected output 2

```
x
y
z
```

Problem 30: Swallowed cancellation

Medium | Debugging task

DEBUGGING TASK - code has valid syntax and correct indentation, but contains a logic/runtime bug

TASK

Fix the worker so cancellation is reported correctly instead of being converted into success. The cancellation handler should not hide asyncio cancellation from the caller.

INPUT FORMAT

Line 1: cancel_after_ms n. Next n lines: job_id delay_ms.
Assume $1 \leq n \leq 20$.

OUTPUT FORMAT

Print job_id OK for jobs completed before cancellation and job_id CANCELLED for the others, preserving input order.

EXPECTED BEHAVIOR

The corrected program should let CancellationError propagate from the worker and convert it to CANCELLED only at the reporting layer.

EXAMPLE INPUT

Example input 1

```
10 3
a 5
b 20
c 30
```

Expected output 1

```
a OK
b CANCELLED
c CANCELLED
```

Example input 2

```
0 2
instant 0
slow 1
```

Expected output 2

```
instant OK
slow CANCELLED
```

Broken code

```
import asyncio
import sys
async def worker(name, delay):
    try:
        await asyncio.sleep(delay / 1000)
        return f"{name} OK"
    except BaseException:
        return f"{name} OK"
async def main():
    cancel_after, n = map(int, sys.stdin.readline().split())
    tasks = []
    for _ in range(n):
        name, delay = sys.stdin.readline().split()
        tasks.append(asyncio.create_task(worker(name, int(delay)
        )))
    await asyncio.sleep(cancel_after / 1000)
    for task in tasks:
        if not task.done():
            task.cancel()
    for task in tasks:
        print(await task)
    asyncio.run(main())
```

Problem 31: Token bucket scheduler

Medium-hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Simulate a token bucket rate limiter for async request starts. The bucket begins full, refills tokens every `refill_ms`, and a request starts as soon as at least one token is available.

INPUT FORMAT

Line 1: `capacity refill_ms n`. Next `n` lines: `request_id arrival_ms`. Assume arrivals are sorted and each request consumes one token.

OUTPUT FORMAT

Print `request_id START start_ms` for each request in input order.

EXAMPLE INPUT

Example input 1

```
2 1 100 5
a 0
b 0
c 0
d 50
e 150
```

Expected output 1

```
a START 0
b START 0
c START 100
d START 200
e START 300
```

Example input 2

```
3 2 100 4
r1 0
r2 0
r3 0
r4 10
```

Expected output 2

```
r1 START 0
r2 START 0
r3 START 0
r4 START 100
```

Problem 32: Async dependency DAG

Medium-hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Run tasks as soon as all their dependencies have completed. Each task has a duration, and independent tasks should run concurrently.

INPUT FORMAT

Line 1: integer n. Next n lines: task_id duration_ms deps, where deps is - or a comma-separated list of task IDs. Assume the dependency graph is acyclic.

OUTPUT FORMAT

Print task_id in completion order; ties use lexicographic task_id order.

EXAMPLE INPUT

Example input 1

```
5
fetch 10 -
auth 5 -
parse 4 fetch
store 3 fetch,auth
notify 1 store
```

Expected output 1

```
auth
fetch
store
notify
parse
```

Example input 2

```
3
a 5 -
b 5 -
c 1 a,b
```

Expected output 2

```
a
b
c
```

Problem 33: Fail-fast gather

Medium-hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Run all jobs concurrently, but fail fast when the first job reports FAIL. Jobs that completed before the failure remain reported; unfinished jobs must be cancelled.

INPUT FORMAT

Line 1: integer n. Next n lines: job_id delay_ms outcome, where outcome is OK or FAIL. Assume ties at the same delay are processed in input order.

OUTPUT FORMAT

Print DONE job_id for successes before the first failure, then FAILED job_id, then CANCELLED count.

EXAMPLE INPUT

Example input 1

```
4
a 5 OK
b 20 OK
c 10 FAIL
d 30 OK
```

Expected output 1

```
DONE a
FAILED c
CANCELLED 2
```

Example input 2

```
3
x 1 OK
y 2 OK
z 3 OK
```

Expected output 2

```
DONE x
DONE y
DONE z
CANCELLED 0
```

Problem 34: Backpressure with bounded queue

Medium-hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Simulate producers feeding a bounded asyncio.Queue and one consumer processing items at a fixed speed. If the queue is full, a producer waits until the consumer frees space.

INPUT FORMAT

Line 1: queue_size process_ms n. Next n lines: item_id produced_at_ms. Assume items are listed by produced_at_ms and ties use input order.

OUTPUT FORMAT

Print item_id CONSUMED at_ms for each item.

EXAMPLE INPUT

Example input 1

```
1 10 4
a 0
b 0
c 0
d 25
```

Expected output 1

```
a CONSUMED 10
b CONSUMED 20
c CONSUMED 30
d CONSUMED 40
```

Example input 2

```
2 5 3
x 0
y 0
z 20
```

Expected output 2

```
x CONSUMED 5
y CONSUMED 10
z CONSUMED 25
```

Problem 35: wait_for cancels a shared task

Medium-hard | Debugging task

DEBUGGING TASK - code has valid syntax and correct indentation, but contains a logic/runtime bug

TASK

Fix the code so a timeout while waiting for a shared task does not cancel the shared task itself. The caller should print TIMEOUT first, then still be able to await the final result later.

INPUT FORMAT

Line 1: delay_ms timeout_ms. The operation takes delay_ms to finish; the first wait uses timeout_ms.

OUTPUT FORMAT

Print TIMEOUT if the first wait times out, then print RESULT when the shared task eventually completes.

EXPECTED BEHAVIOR

The corrected program should protect the shared task, for example with `asyncio.shield`, when applying the timeout.

EXAMPLE INPUT

Example input 1

30 5

Expected output 1

TIMEOUT
RESULT

Example input 2

10 20

Expected output 2

RESULT
RESULT

Broken code

```
import asyncio
import sys
async def fetch(delay):
    await asyncio.sleep(delay / 1000)
    return "RESULT"
async def main():
    delay, timeout = map(int, sys.stdin.readline().split())
    task = asyncio.create_task(fetch(delay))
    try:
        print(await asyncio.wait_for(task, timeout / 1000))
    except asyncio.TimeoutError:
        print("TIMEOUT")
    print(await task)
asyncio.run(main())
```

Problem 36: Bounded async map unordered

Medium-hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Implement an async map that runs at most limit jobs at once and yields results as soon as each job completes. Each result is the square of the input value after its delay.

INPUT FORMAT

Line 1: limit n. Next n lines: value delay_ms. Assume $1 \leq \text{limit} \leq n \leq 100$.

OUTPUT FORMAT

Print value squared_value in completion order.

EXAMPLE INPUT

Example input 1

```
2 4
3 30
4 5
5 5
6 1
```

Expected output 1

```
4 16
5 25
6 36
3 9
```

Example input 2

```
3 3
2 10
7 0
1 5
```

Expected output 2

```
7 49
1 1
2 4
```

Problem 37: Inactivity batch timer

Medium-hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Group events into batches separated by an inactivity timeout. A new batch starts when the gap between consecutive event times is greater than timeout_ms.

INPUT FORMAT

Line 1: timeout_ms n. Next n lines: event_time_ms event_id. Assume events are sorted by time and have unique IDs.

OUTPUT FORMAT

For each batch, print BATCH followed by the event IDs in that batch.

EXAMPLE INPUT

Example input 1

```
10 5
0 a
5 b
20 c
21 d
40 e
```

Expected output 1

```
BATCH a b
BATCH c d
BATCH e
```

Example input 2

```
3 4
0 x
3 y
7 z
8 w
```

Expected output 2

```
BATCH x y
BATCH z w
```

Problem 38: Two-stage worker pipeline

Medium-hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Build a two-stage pipeline with separate worker pools for download and parse stages. A job may enter parsing only after its download completes.

INPUT FORMAT

Line 1: download_workers parse_workers n. Next n lines: job_id download_ms parse_ms. Assume workers choose the next available input-order job when free.

OUTPUT FORMAT

Print job_id when parsing completes.

EXAMPLE INPUT

Example input 1

```
2 1 3
a 5 20
b 10 1
c 1 5
```

Expected output 1

```
a
c
b
```

Example input 2

```
1 2 3
x 5 5
y 1 20
z 1 1
```

Expected output 2

```
x
z
y
```

Problem 39: Supervisor cancellation tree

Medium-hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Simulate a supervisor that runs named child tasks concurrently. If any child fails, the supervisor reports successes that already completed, reports the failure, and cancels all unfinished children.

INPUT FORMAT

Line 1: integer n . Next n lines: child_id delay_ms outcome , where outcome is OK or FAIL. Assume one or more children may fail; use the first failure by completion time.

OUTPUT FORMAT

Print OK child_id for completed successes before the first failure, FAIL child_id , then CANCEL child_id for each unfinished child in input order.

EXAMPLE INPUT

Example input 1

```
4
a 5 OK
b 30 OK
c 10 FAIL
d 1 OK
```

Expected output 1

```
OK d
OK a
FAIL c
CANCEL b
```

Example input 2

```
3
x 5 FAIL
y 5 OK
z 9 OK
```

Expected output 2

```
FAIL x
CANCEL y
CANCEL z
```

Problem 40: Lock release on failure

Medium-hard | Debugging task

DEBUGGING TASK - code has valid syntax and correct indentation, but contains a logic/runtime bug

TASK

Fix the critical section so the lock is released even when a job fails. The program should report failures without breaking later jobs that also need the lock.

INPUT FORMAT

Line 1: integer n. Next n lines: job_id outcome, where outcome is OK or FAIL. Assume jobs must enter the critical section in input order.

OUTPUT FORMAT

Print job_id OK for successful jobs and job_id ERROR for failed jobs, preserving input order.

EXPECTED BEHAVIOR

The corrected program should release the lock in all paths, usually by using async with lock or a try/finally block.

EXAMPLE INPUT

Example input 1

```
3
a OK
b FAIL
c OK
```

Expected output 1

```
a OK
b ERROR
c OK
```

Example input 2

```
2
x FAIL
y OK
```

Expected output 2

```
x ERROR
y OK
```

Broken code

```
import asyncio
import sys
lock = asyncio.Lock()
async def handle(name, outcome):
    await lock.acquire()
    if outcome == "FAIL":
        raise RuntimeError(name)
    await asyncio.sleep(0)
    lock.release()
    return f"{name} OK"
async def main():
    n = int(sys.stdin.readline())
    rows = [sys.stdin.readline().split() for _ in range(n)]
    for name, outcome in rows:
        try:
            print(await handle(name, outcome))
        except RuntimeError:
            print(f"{name} ERROR")
    asyncio.run(main())
```

Problem 41: Per-domain dynamic rate limits

Hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Schedule requests with a separate per-second limit for each domain. Requests are processed in input order, but each domain has its own waiting queue and capacity.

INPUT FORMAT

Line 1: d n . Next d lines: domain limit_per_second. Next n lines: request_id domain arrival_second. Assume arrivals are sorted and each request consumes one slot in its domain's second.

OUTPUT FORMAT

Print request_id START second in input order after scheduling.

EXAMPLE INPUT

Example input 1

```
2 5
api 2
cdn 1
r1 api 0
r2 api 0
r3 api 0
r4 cdn 0
r5 cdn 0
```

Expected output 1

```
r1 START 0
r2 START 0
r3 START 1
r4 START 0
r5 START 1
```

Example input 2

```
1 4
db 1
a db 2
b db 2
c db 3
d db 3
```

Expected output 2

```
a START 2
b START 3
c START 4
d START 5
```

Problem 42: Earliest-deadline async scheduler

Hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Simulate a scheduler with a fixed number of workers. Whenever a worker is free, it chooses the available job with the earliest deadline, breaking ties by input order.

INPUT FORMAT

Line 1: workers n . Next n lines: job_id arrival_ms duration_ms deadline_ms. Assume arrivals are sorted by arrival_ms.

OUTPUT FORMAT

Print job_id FINISH finish_ms OK if finish_ms $\leq$ deadline_ms, otherwise job_id FINISH finish_ms LATE.

EXAMPLE INPUT

Example input 1

```
1 3
a 0 5 20
b 0 10 12
c 6 1 8
```

Expected output 1

```
b FINISH 10 OK
c FINISH 11 LATE
a FINISH 16 OK
```

Example input 2

```
2 3
x 0 10 15
y 0 3 4
z 2 2 10
```

Expected output 2

```
y FINISH 3 OK
z FINISH 5 OK
x FINISH 10 OK
```

Problem 43: Async circuit breaker simulation

Hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Implement a circuit breaker for one service. After threshold consecutive failures, the circuit opens for cooldown calls; after that, one half-open trial decides whether it closes again or reopens.

INPUT FORMAT

Line 1: threshold cooldown n. Next n lines: call_id outcome, where outcome is OK or FAIL. Assume cooldown is counted in skipped calls, not milliseconds.

OUTPUT FORMAT

For each call, print call_id CALL result, call_id SKIP OPEN, or call_id TRIAL result.

EXAMPLE INPUT

Example input 1

```
2 2 7
a FAIL
b FAIL
c OK
d OK
e FAIL
f OK
g OK
```

Expected output 1

```
a CALL FAIL
b CALL FAIL
c SKIP OPEN
d SKIP OPEN
e TRIAL FAIL
f SKIP OPEN
g SKIP OPEN
```

Example input 2

```
2 1 5
a FAIL
b FAIL
c OK
d OK
e FAIL
```

Expected output 2

```
a CALL FAIL
b CALL FAIL
c SKIP OPEN
d TRIAL OK
e CALL FAIL
```

Problem 44: Adaptive concurrency batches

Hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Process jobs in batches using an adaptive concurrency limit. After each batch, increase the limit by 1 if the batch average latency is at most `target_ms`; otherwise decrease it by 1, but never below 1.

INPUT FORMAT

Line 1: `initial_limit target_ms n`. Next `n` lines: `job_id delay_ms`. Jobs are taken in input order, and each batch size is the current limit.

OUTPUT FORMAT

For each completed batch, print `BATCH job_ids AVG average_ms NEXT next_limit`. Use integer floor division for `average_ms`.

EXAMPLE INPUT

Example input 1

```
2 10 5
a 5
b 15
c 4
d 4
e 30
```

Expected output 1

```
BATCH a b AVG 10 NEXT 3
BATCH c d e AVG 12 NEXT 2
```

Example input 2

```
1 5 3
x 1
y 1
z 10
```

Expected output 2

```
BATCH x AVG 1 NEXT 2
BATCH y z AVG 5 NEXT 3
```

Problem 45: Semaphore leak on exception

Hard | Debugging task

DEBUGGING TASK - code has valid syntax and correct indentation, but contains a logic/runtime bug

TASK

Fix the semaphore handling so a failed job does not permanently consume a permit. The corrected program should keep running later jobs after reporting an error.

INPUT FORMAT

Line 1: limit n. Next n lines: job_id outcome, where outcome is OK or FAIL. Assume $1 \leq \text{limit} \leq n \leq 30$.

OUTPUT FORMAT

Print job_id OK for successes and job_id ERROR for failures in completion order.

EXPECTED BEHAVIOR

The corrected program should release the semaphore in a finally block or use `async with sem` around the protected work.

EXAMPLE INPUT

Example input 1

```
1 3
a OK
b FAIL
c OK
```

Expected output 1

```
a OK
b ERROR
c OK
```

Example input 2

```
2 4
w FAIL
x OK
y OK
z OK
```

Expected output 2

```
w ERROR
x OK
y OK
z OK
```

Broken code

```
import asyncio
import sys
async def run_job(name, outcome, sem):
    await sem.acquire()
    if outcome == "FAIL":
        raise RuntimeError(name)
    await asyncio.sleep(0)
    sem.release()
    return f"{name} OK"
async def main():
    limit, n = map(int, sys.stdin.readline().split())
    sem = asyncio.Semaphore(limit)
    rows = [sys.stdin.readline().split() for _ in range(n)]
    for name, outcome in rows:
        try:
            print(await run_job(name, outcome, sem))
        except RuntimeError:
            print(f"{name} ERROR")
    asyncio.run(main())
```

Problem 46: Concurrent log stream merger

Hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Merge several asynchronous log streams by the time each line arrives. Each stream emits its own lines in order, but different streams interleave based on arrival delay.

INPUT FORMAT

Line 1: integer s , the number of streams. For each stream: a line $stream_id\ m$, followed by m lines $arrival_ms\ message$. Assume messages have no spaces and ties use stream order, then line order.

OUTPUT FORMAT

Print $stream_id\ message$ for every log line in global arrival order.

EXAMPLE INPUT

Example input 1

```
2
api 3
0 start
20 ready
30 done
db 2
5 connect
25 query
```

Expected output 1

```
api start
db connect
api ready
db query
api done
```

Example input 2

```
3
a 1
10 A
b 1
0 B
c 1
10 C
```

Expected output 2

```
b B
a A
c C
```

Problem 47: Cancellation-safe pipeline summary

Hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Run jobs through fetch and write stages, then cancel the pipeline at `cancel_after_ms`. Completed writes count as done; jobs in earlier stages or waiting queues must be reported by stage.

INPUT FORMAT

Line 1: `cancel_after_ms fetch_workers write_workers n`. Next `n` lines: `job_id fetch_ms write_ms`. Assume jobs enter fetch in input order.

OUTPUT FORMAT

Print `DONE job_id` for completed writes in completion order. Then print `PENDING fetch_count write_count waiting_count`.

EXAMPLE INPUT

Example input 1

```
15 2 1 3
a 5 5
b 10 10
c 20 1
```

Expected output 1

```
DONE a
PENDING 1 1 0
```

Example input 2

```
8 1 1 3
x 5 10
y 1 1
z 1 1
```

Expected output 2

```
PENDING 0 1 2
```

Problem 48: Total timeout budget with retries

Hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Each operation has a total time budget shared across all retry attempts. Run operations concurrently, but an operation must stop once the next attempt would exceed its remaining budget.

INPUT FORMAT

Line 1: max_attempts n. Next n lines: op_id budget_ms delays outcomes, where delays and outcomes are comma-separated. Outcomes use F and O and align with delays.

OUTPUT FORMAT

Print op_id OK attempts_used elapsed_ms or op_id TIMEOUT attempts_used elapsed_ms in input order.

EXAMPLE INPUT

Example input 1

```
3 3
a 30 5,5 0,0
b 12 5,10 F,0
c 8 3,3,3 F,F,0
```

Expected output 1

```
a OK 1 5
b TIMEOUT 1 5
c TIMEOUT 2 6
```

Example input 2

```
2 2
x 10 6,3 F,0
y 5 5 0
```

Expected output 2

```
x OK 2 9
y OK 1 5
```

Problem 49: Bounded async crawler

Hard | Implementation task

IMPLEMENTATION TASK - write a complete async Python solution

TASK

Implement a bounded async crawler over a supplied graph. Fetches for the same host must respect a per-host limit of one, and the crawler should not visit nodes deeper than max_depth.

INPUT FORMAT

Line 1: start_url max_depth n. Next n lines: url host delay_ms links, where links is - or comma-separated URLs. Assume every linked URL appears in the input.

OUTPUT FORMAT

Print VISIT url in fetch completion order; ties use discovery order.

EXAMPLE INPUT

Example input 1

```
root 2 5
root api 5 a,b
a api 5 c
b cdn 1 d
c api 1 -
d cdn 1 -
```

Expected output 1

```
VISIT root
VISIT b
VISIT d
VISIT a
VISIT c
```

Example input 2

```
home 1 3
home h 1 x,y
x h 1 -
y c 1 -
```

Expected output 2

```
VISIT home
VISIT x
VISIT y
```

Problem 50: Duplicate async cache fetches

Hard | Debugging task

DEBUGGING TASK - code has valid syntax and correct indentation, but contains a logic/runtime bug

TASK

Fix the async cache so concurrent requests for the same key share one in-flight fetch. The current code is runnable, but it performs duplicate fetches during a race.

INPUT FORMAT

Line 1: integer n. Next n lines: key. Assume $1 \leq n \leq 100$ and keys contain no spaces.

OUTPUT FORMAT

Print one value per input key in input order, then print FETCHES distinct_key_count.

EXPECTED BEHAVIOR

The corrected program should protect cache misses with a lock or per-key in-flight task so only one fetch happens per distinct key.

EXAMPLE INPUT

Example input 1

```
4
user
user
team
user
```

Expected output 1

```
USER
USER
TEAM
USER
FETCHES 2
```

Example input 2

```
3
a
b
a
```

Expected output 2

```
A
B
A
FETCHES 2
```

Broken code

```
import asyncio
import sys
cache = {}
fetches = 0
async def get_value(key):
    global fetches
    if key not in cache:
        fetches += 1
        await asyncio.sleep(0)
        cache[key] = key.upper()
    return cache[key]
async def main():
    n = int(sys.stdin.readline())
    keys = [sys.stdin.readline().strip() for _ in range(n)]
    values = await asyncio.gather(*(get_value(k) for k in keys))
    for value in values:
        print(value)
    print("FETCHES", fetches)
asyncio.run(main())
```